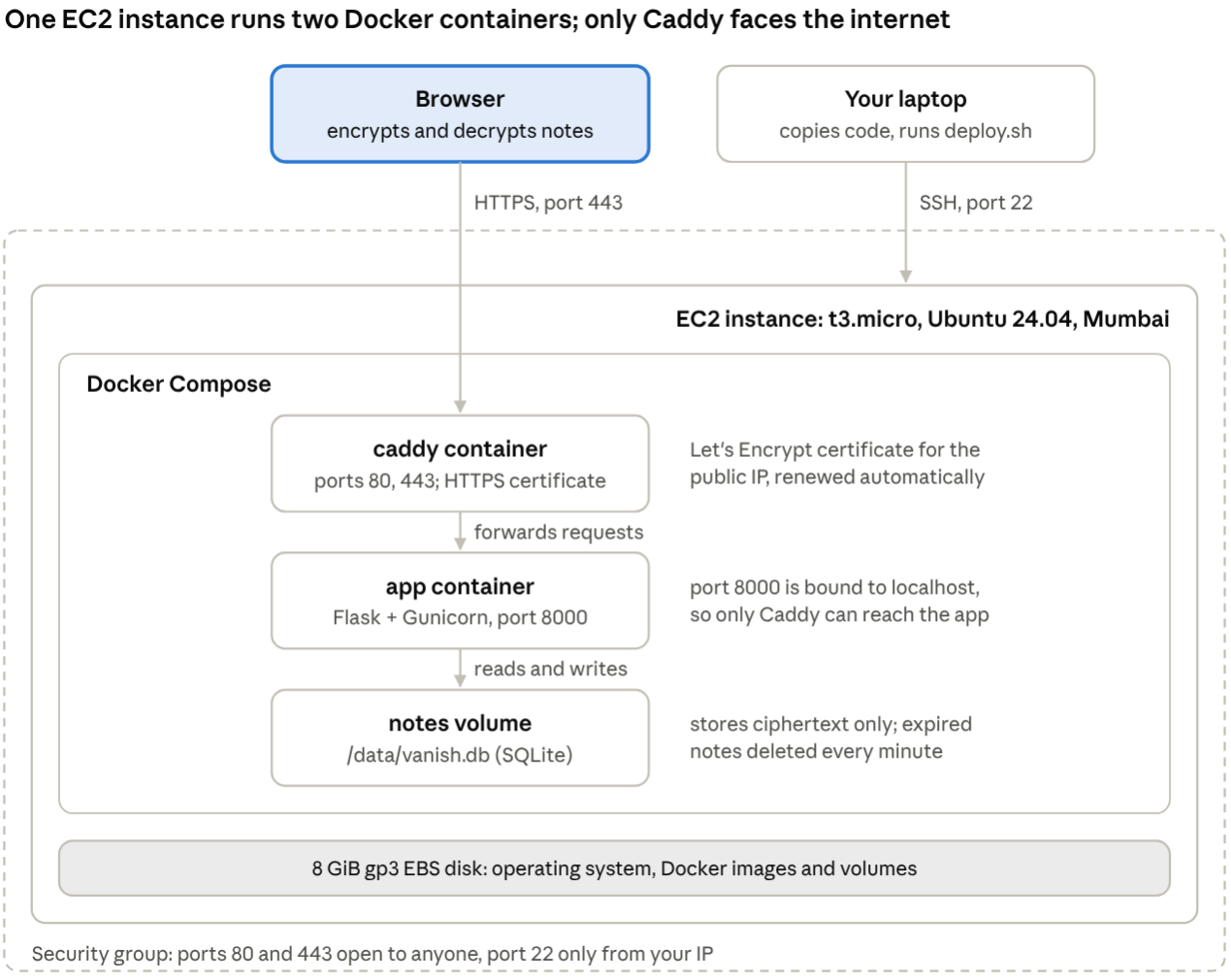
Vanish on EC2: End-to-End Guide and Video Script

Nov 9, 2026 · @Kartik

Overview

Your team will package **Vanish**, a burn-after-reading notes app, in Docker and run it on one Amazon EC2 instance. Following this guide end to end takes about a week; the deploy itself takes about 30 minutes.

A user writes a secret, chooses "destroy after N views or T time", and gets a link. The note is encrypted in the browser with AES-256-GCM before upload. The key is only in the part of the link after `#`, which browsers never send to servers, so the server stores text it cannot read.



Vanish on EC2: one instance, two containers, one volume

Visitors reach only Caddy, over HTTPS; Caddy forwards to the app container, which keeps notes in a SQLite file on a Docker volume. You reach the instance over SSH to deploy.

Piece	Job in Vanish	What you demonstrate
EC2 instance (t3.micro)	Runs Docker and both containers	Instance types, AMIs, user data, instance metadata, monitoring
Security group	Firewall in front of the instance	Inbound rules: 80 and 443 open, 22 limited to your IP
Key pair	SSH login	Public-key authentication instead of passwords
EBS volume (8 GiB gp3)	Disk for the OS, images and notes	Block storage that survives stop and start
Docker image (`Dockerfile`)	Packages Python, Flask and the code	Reproducible builds, non-root container, health check
Docker Compose (`compose.yaml`)	Runs `app` and `caddy` together	Multi-container setup, named volumes, port mapping
Caddy (`Caddyfile`)	HTTPS and reverse proxy	Free Let's Encrypt certificate for an IP address, auto-renewed

Before you start

You use two computers. The **Mac** is used once, for about 20 minutes, to build the Docker images and pack the project into one file. Everything after that happens on the **work laptop**.

Computer	What you do there	What it needs
Mac	Step 1 only: build the images, pack `vanish-transfer.zip`	Docker Desktop for Mac (pick *Apple Silicon* or *Intel chip* to match the Mac), 5 GB free disk

Computer	What you do there	What it needs
Work laptop (Windows, macOS or Linux)	Steps 2–11: AWS, EC2, deploy, report, slides, video	A terminal with `ssh` and `scp` (built into Windows 10/11 PowerShell, macOS and Linux), a PDF viewer, OBS Studio. Optional: Docker Desktop to run the app locally; an Overleaf account to edit the slides
Between them	Carry one file, about 70–100 MB	A USB drive formatted exFAT, or Google Drive

Check which chip the Mac has: Apple menu → **About This Mac**. *Chip: Apple M1/M2/M3/M4* means Apple Silicon; *Processor: Intel* means Intel. The build script handles both.

What it costs

On a new account's **Free plan**, AWS gives $100 of credit and `t3.micro` is Free Tier eligible (AWS EC2 docs). The instance is paid from that credit, so you pay ₹0.

Item	Rate (Mumbai)	Per month, running 24/7
t3.micro instance	$0.0112 per hour (Holori)	~$8.20
Public IPv4 address	~$0.005 per hour	~$3.65
8 GB gp3 disk	~$0.09 per GB-month	~$0.75
Total, taken from the $100 credit		**~$12.60**

Stop the instance when you aren't using it to make the credit last; only the disk keeps charging while it's stopped. The Free plan account closes after 6 months, so finish within that window. On an older account with no credit left, the same setup bills about ₹1,100 a month to your card.

Who owns what

Member	Owns	Report sections	Video scenes
A, backend	`app/main.py`, `tests/`	Implementation (API, SQLite, cleanup), testing	1, 4, 6

Member	Owns	Report sections	Video scenes
B, frontend	`app/static`, `app/pages`, `slides/`, live demo	Implementation (encryption, UI), conclusion	2, 4, 6, 7
C, cloud	`Dockerfile`, `compose.yaml`, `Caddyfile`, `scripts/`, `deploy/`, the AWS account	Architecture, EC2 and Docker, cost, results	3, 5, 6

Suggested week

1. Day 1: build and pack on the Mac (Step 1), move the zip to the work laptop (Step 2), create the AWS account (Step 3).
2. Day 2: launch EC2 and deploy (Steps 4 and 5); all three test it (Step 6).
3. Day 3: collect the screenshots (Step 7) and put your names into the slides.
4. Days 4 and 5: write the report (Step 8).
5. Day 6: rehearse with the slides twice, with a timer (Steps 9 and 10).
6. Day 7: record, edit, submit, then stop the instance (Step 11).

Step 1: On the Mac, build the images and pack everything (20 minutes)

This step ends with one file, `vanish-transfer.zip`, on the Mac's Desktop. It holds the code, the slides, this guide, and both Docker images already built for EC2.

1.1 Install Docker Desktop

1. Go to docker.com/products/docker-desktop and download the Mac version that matches the chip: **Apple Silicon** or **Intel chip**.
2. Open the downloaded `Docker.dmg` and drag **Docker** into **Applications**.
3. Open Docker from Applications. Accept the agreement; you can skip signing in.
4. Wait until the Docker Desktop window shows **Engine running** (bottom left), about 1 minute.
5. Open **Terminal** (press Cmd+Space, type `Terminal`, press Return) and check both commands print a version:

```
docker --version
docker buildx version
```

1.2 Put the project on the Mac

1. Download `vanish.zip` from this chat into **Downloads**.
2. In Terminal, unzip it onto the Desktop and go into it:

```
cd ~/Desktop
unzip ~/Downloads/vanish.zip
cd vanish
ls
```

You should see `app`, `deploy`, `docs`, `scripts`, `slides`, `tests`, `Dockerfile`, `compose.yaml` and `Caddyfile`.

1.3 Build and pack (one command)

```
bash scripts/build-for-ec2.sh
```

The script prints five stages. The first run downloads about 150 MB.

Stage	What happens	Time
1/5	Builds `vanish-app:1.0` for **linux/amd64**, the processor type of a t3.micro	3–6 min on Apple Silicon (emulated), 1–2 min on Intel
2/5	Downloads `caddy:2.11` for linux/amd64	under 1 min
3/5	Checks both images are linux/amd64	seconds
4/5	Saves both images into `dist/vanish-images.tar.gz`	under 1 min
5/5	Zips the whole folder into `~/Desktop/vanish-transfer.zip`	seconds

It's finished when it prints `Done. Copy this one file to the other laptop`. If it stops with `ERROR`, the message says what to fix; the usual one is Docker Desktop not running.

Why linux/amd64: Apple Silicon Macs build arm64 images by default, and an arm64 image fails on a t3.micro with `exec format error`. The script always builds for amd64, and `deploy.sh` double-checks this on the server.

1.4 Optional: check the image starts

```
docker compose up app
```

Open http://localhost:8000, create a note, then press Ctrl+C in Terminal. On Apple Silicon, Docker warns that the image platform (linux/amd64) doesn't match the host; that's expected, it runs under emulation.

1.5 Copy the file off the Mac

- **USB drive**: plug it in, drag `vanish-transfer.zip` from the Desktop onto the drive in Finder, then eject the drive. Use a drive formatted *exFAT* so Windows can read it.
- **Google Drive**: open drive.google.com, choose **New → File upload**, pick `vanish-transfer.zip`.

You're done with the Mac.

Step 2: Set up the work laptop (10 minutes)

1. Create a folder for the project: `C:\Users\<you>\vanish-project` on Windows, or `~/vanish-project` on macOS or Linux.
2. Copy `vanish-transfer.zip` from the USB drive or Google Drive into that folder. Keep this zip as it is: you upload it to EC2 in Step 5.
3. Unzip a copy to read and edit the files. On Windows, right-click the zip → **Extract All** → **Extract** (this creates `vanish-transfer\vanish`). On macOS, double-click it.
4. Open a terminal **in the `vanish-project` folder**. On Windows, open the folder in File Explorer, click the address bar, type `powershell` and press Enter. On macOS, open Terminal and run `cd ~/vanish-project`.
5. Check SSH is available; it should print a version such as `OpenSSH_9.5`:

   ```
   ssh -V
   ```

6. Find these files in the unzipped folder; you'll need them later:

File	Used in
`deploy/user-data.sh`	Step 4: pasted into the EC2 launch form
`slides/vanish-slides.pdf`	Step 10: the slides you present and record
`slides/vanish-slides-notes.pdf`	Step 10: the narration for each slide
`slides/vanish-slides.tex`	Edit names and college, then rebuild on Overleaf
`docs/Vanish_EC2_Docker_Guide.pdf`	This guide, offline
`README.md`	Run and deploy instructions for the report appendix

Optional, to run the app on this laptop as well: install Docker Desktop, open a terminal inside the unzipped `vanish` folder, and run `docker load -i dist/vanish-images.tar.gz`, then `docker compose up app`, and open http://localhost:8000.

Step 3: Create the AWS account (15 minutes)

Member C does this once, with a team email all three can reach. Everything later happens in the web console and over SSH, so no access keys or AWS CLI are needed.

1. Go to aws.amazon.com/free and choose **Create a free account**.
2. When asked to pick a plan, choose **Free plan**. On this plan AWS can't charge your card; usage comes out of the $100 credit.
3. Enter a card for identity verification. Indian cards often see a small refundable verification charge.
4. Finish phone verification and pick **Basic support** (free).
5. Sign in, open the account menu (top right) → **Security credentials** → **Assign MFA device**, and pair an authenticator app.
6. In the region menu (top right), choose **Asia Pacific (Mumbai) ap-south-1**. Use this region for everything below.

Step 4: Launch the EC2 instance (10 minutes)

One instance runs the whole app. In the console search bar type **EC2**, open it, and choose **Launch instance**. Fill in the form as below and leave everything else at its default.

Setting	Value	Why
Name	`vanish`	Shows in the instance list
Application and OS Image (AMI)	**Ubuntu Server 24.04 LTS**, 64-bit (x86)	Free Tier eligible; the user-data script is written for it
Instance type	**t3.micro** (2 vCPU, 1 GiB)	Free Tier eligible on new accounts
Key pair	**Create new key pair** → name `vanish-key`, type RSA, format `.pem`	The file downloads once; it's your only way to SSH in
Network settings → Allow SSH traffic from	**My IP**	Only your current internet connection can log in
Allow HTTPS traffic from the internet	Ticked	Visitors reach the app on port 443
Allow HTTP traffic from the internet	Ticked	Let's Encrypt checks port 80 before issuing the certificate; Caddy also redirects HTTP to HTTPS
Configure storage	8 GiB gp3 (default)	Room for the OS, Docker images and notes
Advanced details → User data	Paste the whole of `deploy/user-data.sh`	Installs Docker and adds 1 GB of swap at first boot

Then:

1. Choose **Launch instance**, then **View all instances**.
2. Wait until *Instance state* is **Running** and *Status check* shows **3/3 checks passed** (2 to 3 minutes).
3. Select the instance and copy its **Public IPv4 address**, for example `13.233.45.67`. You'll use it in every step below.
4. Move `vanish-key.pem` from Downloads into `vanish-project`, next to `vanish-transfer.zip`. Then restrict who can read it, or SSH will refuse it:

```
# macOS or Linux
chmod 400 vanish-key.pem
```

```
# Windows PowerShell, inside vanish-project
icacls vanish-key.pem /inheritance:r /grant:r "${env:USERNAME}:R"
```

To paste the user data in the launch form: open `deploy/user-data.sh` from the unzipped folder in a text editor (Notepad, TextEdit or VS Code), press Ctrl+A then Ctrl+C (Cmd on macOS), and paste into the **User data** box.

The wizard creates a security group named `launch-wizard-1` with three inbound rules: SSH (22) from your IP, HTTP (80) and HTTPS (443) from anywhere. You can see it under the instance's **Security** tab; take a screenshot for the report.

Step 5: Upload the zip and deploy (10 minutes)

Run these in the terminal opened in `vanish-project`, which holds `vanish-transfer.zip` and `vanish-key.pem`. Replace `13.233.45.67` with your instance's public IP everywhere. The commands are the same on Windows PowerShell, macOS and Linux.

1. Upload the zip (about 70–100 MB, 1 to 3 minutes). Type `yes` when asked about the host fingerprint the first time.

   ```
   scp -i vanish-key.pem vanish-transfer.zip ubuntu@13.233.45.67:~
   ```

2. Log in to the instance:

   ```
   ssh -i vanish-key.pem ubuntu@13.233.45.67
   ```

   The prompt changes to `ubuntu@ip-172-31-...:~$`. Everything below runs on the instance.

3. Check the first-boot script finished. It should print `Docker is ready`; if the file is missing, wait a minute and try again.

   ```
   cat ~/READY.txt
   ```

4. Unzip and deploy:

   ```
   unzip -q vanish-transfer.zip
   cd vanish
   bash deploy/deploy.sh
   ```

   Expected output includes `Loaded image: vanish-app:1.0`, `Loaded image: caddy:2.11`, `Starting Vanish at https://13.233.45.67`, and a table with `vanish-app-1` and `vanish-caddy-1` both **Up**.

5. Watch Caddy get the HTTPS certificate. Wait for `certificate obtained successfully`, then press Ctrl+C:

   ```
   docker compose logs -f caddy
   ```

If you see `permission denied`, put `sudo` in front, or log out with `exit` and SSH in again once.

6. On the laptop, open **https://13.233.45.67**. You should see "Share a secret that deletes itself." with a padlock.
7. Type `exit` to leave the instance. The app keeps running.

What deploy.sh does

Stage	What happens
Find the address	Asks the EC2 instance metadata service (IMDSv2) for the public IP and writes `SITE_ADDRESS=<ip>` to `.env`
Load the images	`docker load -i dist/vanish-images.tar.gz`; nothing is built or downloaded on the instance
Check the processor	The image must be amd64 on a t3.micro; otherwise it stops and says how to fix it
Start	`docker compose up -d --no-build`: `caddy` on ports 80 and 443, `app` on 127.0.0.1:8000

The `notes` volume keeps the database and `caddy_data` keeps the certificate, so both survive restarts. Browsers only allow the in-browser encryption on HTTPS pages, which is why Caddy is part of the setup.

If the certificate keeps failing, use a free domain name instead: create one at duckdns.org pointing to your IP, then run `bash deploy/deploy.sh yourname.duckdns.org`.

If you change the code later

- **Quickest:** edit the file on the instance (for example `nano app/static/style.css`), then run `bash deploy/deploy.sh --build`. This builds the image on the instance in 2 to 4 minutes.
- **Or rebuild elsewhere:** on any computer with Docker and bash (macOS, Linux, or WSL on Windows), run `bash scripts/build-for-ec2.sh`, upload the new `vanish-transfer.zip` with `scp`, then on the instance run `unzip -oq vanish-transfer.zip && cd vanish && bash deploy/deploy.sh`.

Step 6: Test the live app

Run these ten checks on `https://<your IP>`, ideally with all three members watching. Anything that fails goes to Troubleshooting below.

#	Do this	Expected result
1	Open `https://<your IP>`	Create page loads with a padlock
2	Open `http://<your IP>`	Redirects to `https://`
3	Write a note, set 2 views and 1 hour, press **Create secret link**	Text dissolves; link and QR code appear
4	Open the link in a private window	"Someone sent you a secret" with a live countdown
5	Press **Open note**	Note appears; "1 more time"; the `#key` part disappears from the address bar
6	Open the link a second time, then a third	Second: "That was the last view". Third: "This note has vanished"
7	Create a note with a passphrase; open it on your phone via the QR code; try a wrong passphrase, then the right one	Wrong one is rejected; right one shows the note
8	Create a note, press **Destroy now**, then open its link	"This note has vanished"
9	Delete everything after `#` in a fresh link and open it	"This link is missing its key"; no view is used
10	On the instance: `docker compose restart`, then reload a note created earlier	The note is still there (data lives in the volume)

Two checks for the report:

- [ ] The database holds only ciphertext. On the instance, after creating a note with known text:

```
docker compose exec app python -c "import sqlite3;
print(sqlite3.connect('/data/vanish.db').execute('select id,
substr(ciphertext,1,60), view_count, max_views from notes').fetchall())"
```

- [ ] The app port is private. From your laptop, `http://<your IP>:8000` must time out.

Step 7: Collect screenshots for the report

Take these 15 screenshots while the app is live and save them as `fig01.png` to `fig15.png` in a shared folder. Blur the account ID and IP address where they appear, if you prefer.

Fig	Capture	Where	Report section
01	Create page with a note typed and options set	the app	Results
02	Link ready, with QR code	after **Create secret link**	Results
03	Recipient page with the countdown	link in a private window	Results
04	Passphrase prompt with the wrong-passphrase error	phone or browser	Results
05	The revealed note, "That was the last view"	the link	Results
06	"This note has vanished"	the link again	Results
07	Padlock → certificate details showing Let's Encrypt and the IP	browser address bar	Implementation
08	Instance summary: t3.micro, running, public IP, AMI	**EC2 → Instances**	Cloud services
09	Security group inbound rules: 22, 80, 443	instance **Security** tab	Security
10	Storage: 8 GiB gp3 EBS volume	instance **Storage** tab	Cloud services
11	CPU utilisation graph	instance **Monitoring** tab	Results
12	`docker compose ps` showing both containers healthy	SSH terminal	Implementation
13	`docker compose logs caddy` with "certificate obtained successfully"	SSH terminal	Implementation
14	The database query from Step 6 showing ciphertext only	SSH terminal	Security
15	`pytest -q` showing 16 passed	your laptop	Results

Take 14 right after creating a note with known text, so the report can say "this is the note 'wifi password…' as the server stores it".

Step 8: Write the report

Aim for 12 to 15 pages including the figures from Step 7. The sections below cover everything the guidelines ask for.

#	Section	Pages	Writer	What it must contain
1	Problem statement	1	A	Passwords and secrets shared in chat stay forever in history, backups and screenshots; existing tools ask you to trust their server.
2	Objectives	1	A	Expire after N views or T time; server can't read notes; correct view limits under concurrent access; containerised; deployed on a public cloud over HTTPS
3	System architecture	2	C	The diagram from the Overview; request flow create → preview → open → expire; figs 08, 09
4	Cloud services used	2	C	EC2 (instance type, AMI, user data, instance metadata), EBS, security groups, key pairs, public IPv4, EC2 monitoring; Docker and Docker Compose; Let's Encrypt via Caddy; fig 10
5	Implementation	3–4	A, B, C	Encryption with the key in the link fragment (B); the atomic `UPDATE … RETURNING` and the race it prevents (A); cleanup thread and rate limit (A); `Dockerfile` layers and non-root user, `compose.yaml`, `Caddyfile`, `deploy.sh`, and building amd64 images on an Apple Silicon Mac (C); figs 07, 12, 13
6	Results	2–3	All	The ten checks from Step 6 with pass marks; figs 01–06, 11, 14, 15; cost table from "Before you start"

#	Section	Pages	Writer	What it must contain
7	Conclusion and future work	1	B	Each objective met; next steps: a custom domain, a managed database, two instances behind a load balancer
–	Appendix	any	C	How to run (copy the README), repository link, contribution table

Copy these from the repo

- Atomic view counting: `open_note` in `app/main.py`.
- Encryption format: the comment at the top of `app/static/crypto.js`.
- Deployment files: `Dockerfile`, `compose.yaml`, `Caddyfile`, `deploy/deploy.sh` are short enough to include whole.
- Security notes and API table: `README.md`.

Contribution table for the appendix

Member	Code owned	Report sections	Video scenes
A	`app/main.py`, `tests/`	1, 2, 5 (backend)	1, 4, 6
B	`app/static`, `app/pages`, `slides/`	5 (frontend), 7	2, 4, 6, 7
C	Docker files, `scripts/`, `deploy/`, AWS	3, 4, 5 (deployment), appendix	3, 5, 6

Step 9: Prepare the slides and record the video

The video is built around the LaTeX slide deck in `slides/`: 22 slides at 16:9, with each slide's narration stored as a speaker note. You record the slides full screen while narrating, and cut to the live app twice: once for the demo, once for the terminal.

9.1 Put your names in the slides (Overleaf, 10 minutes)

1. In the unzipped project, compress the `slides` folder: on Windows right-click it → **Send to → Compressed (zipped) folder**; on macOS right-click → **Compress "slides"**.
2. Sign up free at overleaf.com, then choose **New Project → Upload Project** and pick `slides.zip`.

3. Open `vanish-slides.tex`. In the block marked `EDIT ME`, replace `Member A`, `Member B`, `Member C`, `Your College Name` and `Cloud Computing Project`.
4. Open **Menu**, check **Compiler** is **pdfLaTeX**, press **Recompile**, then download the PDF. Overleaf uses the Fira Sans font, so it looks slightly different from the PDF in the zip.
5. To get the narration version, find the line `% \def\withnotes{}` near the top, delete the `%`, recompile and download: each page now shows the slide on the left and its narration on the right. Put the `%` back afterwards.

With LaTeX installed locally instead, run `pdflatex vanish-slides.tex` twice for the slides, and `pdflatex -jobname=vanish-slides-notes "\def\withnotes{}\input{vanish-slides.tex}"` twice for the notes.

9.2 Tools

Job	Tool	Settings
Record slides and voice	OBS Studio with a **Display Capture** source	MP4, 1920×1080, 30 fps
Show the slides	Any PDF viewer in full-screen mode: Adobe Reader (Ctrl+L), macOS Preview (**View → Slideshow**), or Chrome/Edge (open the PDF, press F11)	One slide per screen
Read the narration	`vanish-slides-notes.pdf` on a phone or second screen	
Phone demo	Built-in screen recorder (Android Quick Settings, iPhone Control Centre)	
Editing	Shotcut, DaVinci Resolve, or Clipchamp on Windows	Join clips, trim pauses
Microphone	Wired earphones with a mic	Much clearer than a laptop mic

If members record from different places, a free Zoom meeting with screen sharing and **Record on this computer** works too: one person shares the slides and each presenter speaks in turn.

9.3 Before recording

- Instance running, app open at `https://<your IP>` with the padlock showing.
- One terminal already logged in over SSH and sitting in `~/vanish`.

- AWS console open on **EC2 → Instances**, your instance selected.
- Demo text ready to paste: `wifi: CoffeeHouse_5G / password: purple-otter-91`.
- Do Not Disturb on; chat apps closed; slides PDF open in full screen on slide 1.

9.4 Record, edit, export

1. Rehearse once with a timer, reading from the notes PDF. If a scene runs long, cut words, not slides.
2. Record each scene in Step 10 as its own clip, by its presenter. Advance slides with the arrow keys as you speak.
3. Record the two cutaways separately: the live demo (Scene 2) and the terminal plus EC2 console (Scene 5).
4. In the editor, place the clips in scene order and insert each cutaway where its scene says.
5. Check the total is between 9:30 and 10:00, then export MP4, H.264, 1080p. If links are accepted, upload to YouTube as *Unlisted* or to Google Drive with "anyone with the link" access.

Step 10: The 10-minute video script

The script follows the 22 slides in order, with two live cutaways. A fuller version of each line is in `vanish-slides-notes.pdf`, marked [A], [B] or [C] for the presenter. Total speech is about 9 minutes 50 seconds; say it in your own voice.

Scene	Time	Presenter	Slides	Topic
1	0:00–1:15	A	1–4	Title, agenda, problem, objectives
2	1:15–3:30	B	5–7, then live demo	What Vanish does
3	3:30–4:30	C	8–9	Architecture and EC2 features
4	4:30–6:15	B, then A	10–12	Encryption, API, atomic view counting
5	6:15–8:00	C	13–16, then terminal	Docker, build once, deploy to EC2
6	8:00–9:00	B, A, C	17–19	Security, testing, cost

Scene	Time	Presenter	Slides	Topic
7	9:00–9:50	B	20–22	Lessons, future work, summary

Scene 1: The problem (0:00–1:15, A)

On screen	Say
Slide 1: title	"Hi, we're A, B and C. Our cloud computing project is Vanish, a web app for sharing a secret that deletes itself. We packaged it with Docker and deployed it on an Amazon EC2 virtual machine."
Slide 2: agenda	"We'll cover the problem, a demo, the architecture, the implementation, how we containerised and deployed it, and what we learned."
Slide 3: the problem	"Think about the last time you sent someone a password. It's still sitting in that chat, in backups, maybe in a screenshot. Messages last forever; secrets shouldn't. And one-time-link websites usually receive your secret in plain text."
Slide 4: objectives	"So we set five objectives: notes self-destruct after N views or T time; the server can never read them; view limits hold under concurrent access; the app is fully containerised; and it runs on a public cloud over HTTPS."

Scene 2: What Vanish does (1:15–3:30, B)

On screen	Say
Slide 5: how it works	"You write a secret, choose a view limit and an expiry, and your browser encrypts it with AES-256 before anything is sent. You share a link or QR code. Each open uses one view, and at the last view or at expiry, it's gone."
Slide 6: creating a secret	"Here's the create page and the result: a link, a QR code, and a summary of the rules."

On screen	Say
Slide 7: receiving it	"The recipient sees a countdown, decrypts the note on their own device, and after the last view, nothing is left."
Cutaway: live demo on `https://<your IP>`	"Let's do it live on our EC2 instance." Paste the Wi-Fi note, choose 2 views and 1 hour, add passphrase `mango`, press **Create secret link**: "Watch the text dissolve: it's being encrypted right here."
Cutaway: phone recording	"My teammate scans the QR code. Opening the page doesn't use a view, so chat previews can't burn it. A wrong passphrase is rejected; the right one shows the note."
Cutaway: laptop, open the link twice more	"Second view: that was the last one. Third try: this note has vanished."

Scene 3: Architecture (3:30–4:30, C)

On screen	Say
Slide 8: architecture	"Everything runs on one EC2 instance: a t3.micro with Ubuntu in Mumbai. A security group lets anyone in on ports 80 and 443, but SSH only from our IP. Inside, Docker Compose runs two containers: Caddy, the public web server with a free HTTPS certificate, and our app, Flask with Gunicorn. Notes live in SQLite on a Docker volume on the EBS disk."
Slide 9: EC2 features	"We used the instance type and Ubuntu AMI, a user-data script that installs Docker at first boot, the instance metadata service to find the public IP, a security group, a key pair for SSH, an EBS volume, and EC2's built-in monitoring."

Scene 4: Implementation (4:30–6:15, B then A)

On screen	Say
Slide 10: zero-knowledge (B)	"The part of the link before the hash tells the server which note to fetch. The part after it is the 256-bit key, and browsers never send that part anywhere. So the server only stores ciphertext. A passphrase adds a second layer using PBKDF2."
Slide 11: API (A)	"The backend is one Python file with five endpoints. Reading a note's details never uses a view; opening one does, atomically. Expired notes are deleted every minute."
Slide 12: atomic view counting (A)	"This single SQL statement adds a view only if one is left and the note hasn't expired, and returns it in the same step. Our test fires eight requests at a one-view note at once: exactly one wins."

Scene 5: Docker and deployment (6:15–8:00, C)

On screen	Say
Slide 13: Dockerfile	"The Dockerfile starts from slim Python, installs dependencies first for fast rebuilds, and runs the app as a non-root user with a health check."
Slide 14: Compose and Caddy	"Compose runs both containers. The app's port is bound to localhost, so only Caddy can reach it. The Caddyfile gets a Let's Encrypt certificate for our IP, because browser encryption only works over HTTPS."
Slide 15: build once	"We built the images once on a Mac, for linux/amd64, because Apple Silicon builds arm64 by default and that won't run on a t3.micro. One script saves both images into a zip that we carried to the deploy laptop."
Slide 16: deploy steps	"Then four steps: launch the instance with our user-data script, copy the zip with scp, unzip, and run deploy.sh."

On screen	Say
Cutaway: terminal	Show `bash deploy/deploy.sh` output: "It finds the public IP, loads the images without building anything, checks the architecture, and starts both containers." Then `docker compose ps`: "Both are up."
Cutaway: EC2 console, **Monitoring** tab	"And EC2's monitoring shows the instance barely uses its CPU."

Scene 6: Security, testing, cost (8:00–9:00, B, A, C)

On screen	Say
Slide 17: security (B)	"For data: ciphertext only, hashed delete tokens, rate limits. For infrastructure: three open ports, a private app port, a non-root container, and strict HTTPS headers."
Slide 18: testing (A)	"All 16 automated tests pass, including the concurrency test, and we checked the full cycle on the live instance."
Slide 19: cost (C)	"Running all month costs about twelve and a half dollars, paid from the AWS Free plan's credit, so we paid nothing."

Scene 7: Wrap-up (9:00–9:50, B)

On screen	Say
Slide 20: lessons	"We learned that CPU architecture matters for images, that browser crypto needs HTTPS, that public IPs change on restart, and that a 1 GB instance needs swap."
Slide 21: future work	"Anyone with the full link can read a note, so use a passphrase for sensitive data. Next we'd add a domain, a managed database and a load balancer."
Slide 22: summary	"Vanish shares secrets that destroy themselves, never sees them in plain text, and runs in Docker on Amazon EC2. Thank you for watching."

Everyday commands and troubleshooting

Run these on the instance, inside `~/vanish`.

Task	Command
See both containers and their health	`docker compose ps`
Follow the app's logs	`docker compose logs -f app`
Follow Caddy's logs (certificates)	`docker compose logs -f caddy`
After the instance was stopped and started (new IP)	`bash deploy/deploy.sh`, then use the new IP
Rebuild on the instance after editing code	`bash deploy/deploy.sh --build`
Use a new zip built elsewhere	upload it with `scp`, then `cd ~ && unzip -oq vanish-transfer.zip && cd vanish && bash deploy/deploy.sh`
Restart both containers	`docker compose restart`
Stop the app, keep the notes	`docker compose down`
Check disk space	`df -h /` and `docker system df`

Troubleshooting

Symptom	Cause	Fix
Mac: `ERROR: Docker is not running`	Docker Desktop closed	Open Docker Desktop, wait for **Engine running**, run the script again
Mac: `docker: 'buildx' is not a docker command`	Old Docker Desktop	Update Docker Desktop (Docker menu → **Check for Updates**)
Mac: stage 1/5 is very slow	Building amd64 on Apple Silicon is emulated	Normal; allow up to 10 minutes the first time
`unzip` on Windows shows `vanish-transfer\vanish\vanish`	Extracted into an extra folder	Use the inner `vanish` folder; the zip uploaded to EC2 is unaffected
`ssh: Permission denied (publickey)`	Wrong user or key	Log in as `ubuntu@`, with `-i vanish-key.pem`, run from the folder that holds the key
`UNPROTECTED PRIVATE KEY FILE`	Key readable by others	macOS/Linux: `chmod 400 vanish-key.pem`. Windows PowerShell: `icacls vanish-key.pem /inheritance:r /grant:r "${env:USERNAME}:R"`
SSH hangs, then times out	Your home or college IP changed	**Security** tab → security group → **Edit inbound rules** → set the SSH rule's source to **My IP** again
`cat ~/READY.txt`: no such file	User data still running, or not pasted	Wait 2 minutes; if still missing, run the commands in `deploy/user-data.sh` by hand with `sudo`
`The image is built for arm64 but this instance is amd64`	Image built without the script, or a t4g instance chosen	Rebuild with `scripts/build-for-ec2.sh`, or run `bash deploy/deploy.sh --build`
`exec format error` in `docker compose logs app`	Same architecture mismatch	As above

Symptom	Cause	Fix
`permission denied ... docker.sock`	Group change needs a fresh login	`exit`, then `ssh` in again (the deploy script already falls back to `sudo`)
Browser shows a certificate warning	Certificate not issued yet	Wait a minute and reload; check `docker compose logs caddy`; port 80 must be open to the internet
Certificate keeps failing	Let's Encrypt can't reach the IP, or IP certificates are refused	Use a free DuckDNS name pointing at the IP: `bash deploy/deploy.sh yourname.duckdns.org`
Page says "Encryption needs a secure connection"	Opened with `http://` on an IP	Use `https://`
Site down after Stop/Start	The public IP changed	Run `bash deploy/deploy.sh` and use the new IP
Overleaf: `File img/create.jpg not found`	The `img` folder wasn't uploaded	Upload the whole `slides` folder as a zip, not just the `.tex` file

Step 11: Submit, then shut down

Submit first. Keep the instance until grading if your examiner may open the live app; stop it otherwise to save credit.

Submission checklist

- [ ] Report PDF with all seven sections, figures 01–15 and the contribution table
- [ ] Source code: the project folder zipped, **without** `dist/`, `.env` or `vanish-key.pem` (the code is small; the images are rebuilt from it)
- [ ] `README.md` included (run and deploy instructions are a required deliverable)
- [ ] Slides: `vanish-slides.pdf` with your names
- [ ] Video: MP4, 1080p, 9:30 to 10:00, all three members speaking
- [ ] Live URL `https://<your IP>` in the report, if the instance stays up for grading

Pause it (keeps everything)

EC2 → **Instances** → select `vanish` → **Instance state → Stop instance**. Only the 8 GB disk is charged while stopped (about $0.75 a month). To resume, **Start instance**, SSH in with the new public IP, and run `bash deploy/deploy.sh`.

Delete it (after grading)

1. EC2 → **Instances** → select `vanish` → **Instance state → Terminate (delete) instance**. This deletes the instance, its disk and all notes.
2. EC2 → **Key pairs** → delete `vanish-key`, and delete the `.pem` file from your laptop.
3. EC2 → **Security groups** → delete `launch-wizard-1` (after the instance has finished terminating).
4. Billing and Cost Management → **Bills**: confirm nothing else is running.